Evaluating the Impact of Quantization on LLM Reasoning

A Systematic Analysis of Precision, Throughput, and Accuracy Tradeoffs on Mathematical Benchmarks

Anna Thomas
Govind Arun Kumar

Problem Statement

Large language models have strong mathematical reasoning capabilities, but deployment constraints demand efficient inference through reduced numerical precision. This project evaluates how inference based post training quantization affects LLM reasoning performance on mathematical problem solving tasks.

Project Objective

- Given a fixed GPU memory and time budget, which precision format yields the best reasoning accuracy for mathematical problem solving?
- Measure the reasoning degradation in INT8 and 4-bit (NF4) models relative to a BF16 baseline on chain-of-thought (CoT) math problems.
- Capture the actual memory consumption, throughput, and latency for each configuration.

Comparison Factors

Models:

- Llama 3.1 8B Instruct: Grouped Query Attention (GQA) architecture.
- Mistral 7B Instruct v0.3: Sliding Window Attention (SWA) architecture.

Quantization Methods:

- BF16 (16-bit, BrainFloat): The native baseline. Maximum accuracy, highest memory/compute cost.
- INT8 (8-bit Integer): LLM.int8() absmax quantization. Significant memory reduction, minimal expected accuracy loss.
- NF4 (4-bit, NormalFloat): QLoRA-style double quantization. Extreme memory reduction, potential reasoning degradation.

Model

Llama 3.1 8B Instruct (Meta):

- 84.5% accuracy on GSM8K (8-shot CoT) , strongest open source 8B model on math reasoning
- Uses Grouped Query Attention (GQA) , the dominant architecture in modern LLMs (GPT-4, Gemma, etc.)
- 128K vocabulary, trained on 15T+ tokens , most extensively trained open 8B model available

Mistral 7B Instruct v0.3 (Mistral AI):

- ~45% on GSM8K (base, 8-shot) , architecturally different baseline that lets us test if quantization robustness is architecture-dependent
- Uses Sliding Window Attention (SWA) , a fundamentally different attention mechanism that processes tokens in fixed-size local windows vs. Llama's global grouped queries
- 32K vocabulary with extended function-calling tokens , structurally different tokenizer means the same math problem gets encoded differently, testing whether quantization sensitivity is tokenization-dependent

Datacard

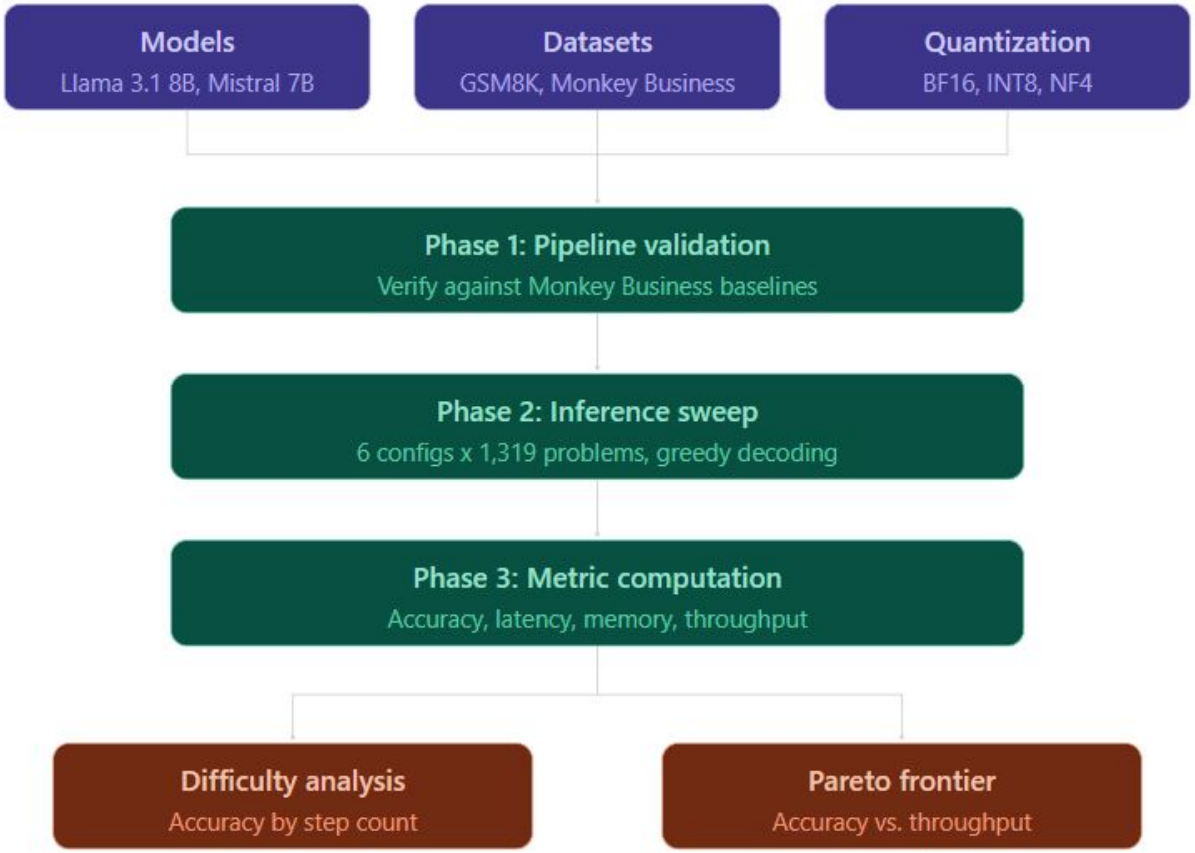
GSM8K (Primary Evaluation):

- 1,319 grade-school math word problems requiring 2 to 8 reasoning steps each
- Every answer is a single number extracted via #### <number> format , fully automatic verification with zero human judgment
- The standard benchmark used in every major LLM technical report (Meta, Mistral, Google, OpenAI) , makes our results directly comparable to published numbers

Monkey Business (Pipeline Validation Only):

- 127 GSM8K problems × 10,000 pre-generated samples from Llama-3-8B-Instruct, each labeled correct/incorrect by Stanford researchers (Brown et al., 2024)
- We do NOT evaluate on this. We use it once, at the start, to verify our prompt template and answer extraction code produce the same accuracy numbers as published results

Project Workflow

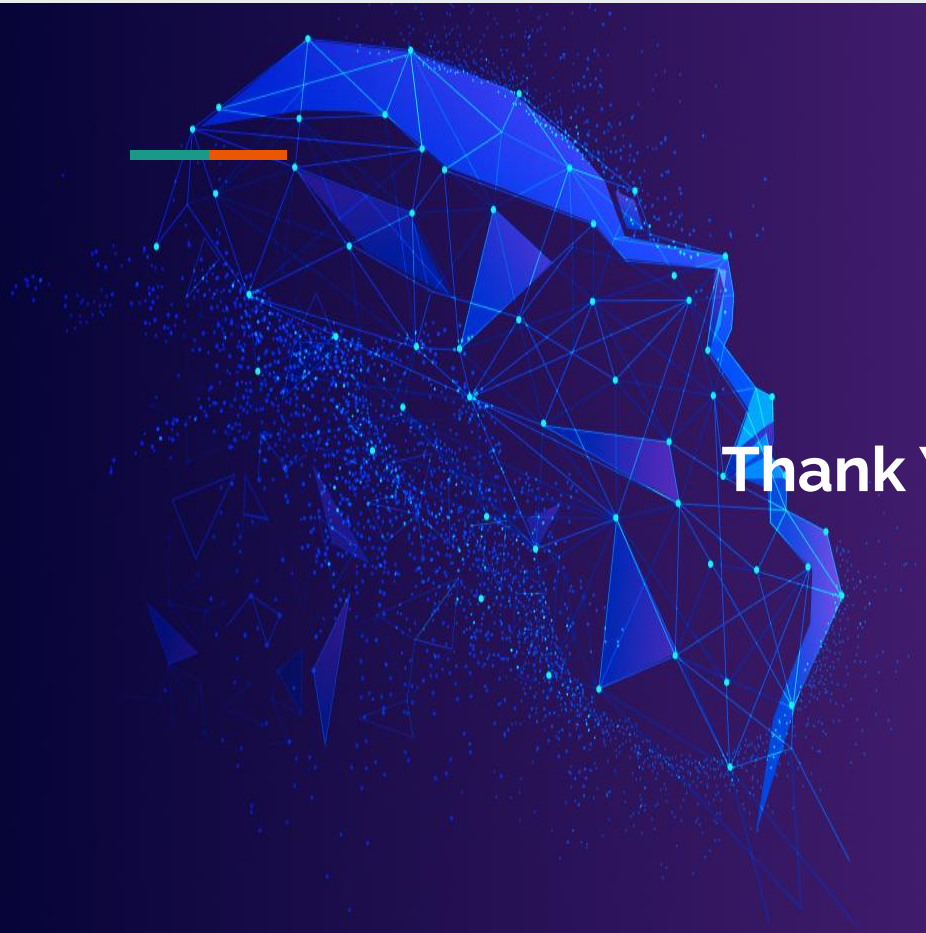


Research Question?

How much reasoning accuracy do LLMs lose when compressed from 16-bit to 8-bit and 4-bit precision, and does the degradation hit harder on multi-step problems than simple ones?.

Expected Results

- Quantization Difficulty Analysis: Map reasoning degradation against problem step count to locate where multi-step logical chains fail.
- Architectural Robustness Comparison: Contrast how Llama (GQA) vs. Mistral (SWA) handle precision reduction.
- Pareto Frontier: A plot of Math Accuracy against GPU Memory/Compute Cost to help developers identify the optimal configuration for their hardware budget.



Thank You